Busca clássica e heurística

Davi Duarte de Paula

Fatec de Rio Claro

2026

Conteúdo da aula

- Princípio de formulação de problemas
- Resolução de problemas via algoritmos de busca
- Busca clássica
- Busca heurística

Resolução de problemas

- Por vezes, a solução de um problema passa pela execução de uma sequência de ações e não de uma única ação do agente.
- Na aula passada vimos o exemplo do aspirador de pó, no qual um conjunto de ações que faz o agente percorrer toda a sala resolvia o problema de limpá-la.
- Veja que nesse exemplo há uma **meta** a ser atingida, que é limpar a sala. Uma vez consideradas as informações disponíveis sobre o ambiente e as limitações do agente, passa-se à **formulação do problema**, que considera quais ações e estados devem ser considerados de forma a atingir a meta.
- A procura por uma sequência de ações que resolve um problema é denominada **busca**.

Resolução de problemas

- Um agente que busca uma sequência de ações para resolver um problema é chamadode **agente resolvedor de problemas**.
- Nesta aula, consideraremos apenas os ambientes do tipo: **episódico, uniagente, totalmente observável, determinístico, estático, discreto e conhecido**.
- Busca informada: É o algoritmo de busca no qual o agente consegue estimar o quão longe está do seu objetivo.
- Busca não informada: É o algoritmo de busca no qual tal estimativa não está disponível.

Mapa simplificado da Romênia

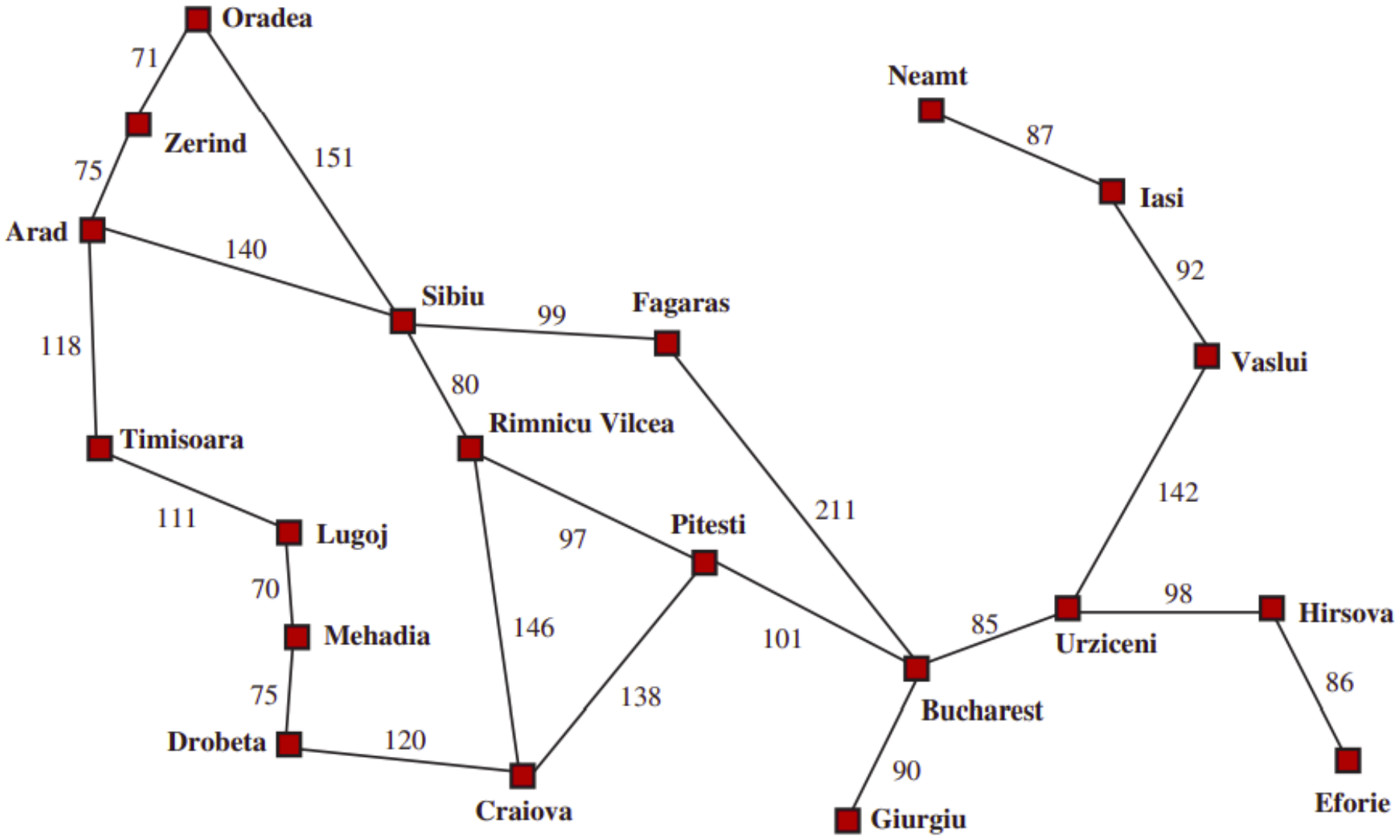

Figure 3.1 A simplified road map of part of Romania, with road distances in miles.

Agentes resolvedores de problemas

- Suponha que um agente esteja na cidade de Arad, na Romênia, de férias. Ele deseja ir para a cidade de Bucharest.
- Entretanto, ao pegar a estrada para sair de Arad, ele observa que existem apenas 3 caminhos:
  - Ir para Zerind
  - Ir para Sibiu
  - Ir para Timisoara
- Nenhum desses caminhos é o objetivo.
- Se o agente não sabe o resultado das suas ações, então o ambiente é **desconhecido**.
- Por exemplo, ele vê que tem 3 cidades adjacentes, mas as estradas não tem placas. Ele não sabe quais estradas levem a quais cidades. Ele também não tem acesso ao mapa.
- Essa é a definição de ambiente **desconhecido**, ou seja, o agente não conhece a função de transição: $Result(s, a)$.
- Se o ambiente é desconhecido, o agente não tem escolha e deve escolher um caminho aleatoriamente.

Agentes resolvedores de problemas

- Nessa aula, vamos sempre considerar que o ambiente é **conhecido**, ou seja, o agente tem acesso ao mapa.
- Ele sabe que, segundo a equação $Result(s, a)$, se $s = arad$ e:
  - $a = irZerind$, então $Result(s, a) =$ Zerind
  - $a = irSibiu$, então $Result(s, a) =$ Sibiu
  - $a = irTimisoara$, então $Result(s, a) =$ Timisoara

Processo de resolução de problemas

- O agente segue esse processo de quatro etapas para resolver o problema em questão (ex.: Rota de Arad para Bucharest):
  - Definição do objetivo: O agente define o objetivo, ex: Chegar em Bucharest. Pode haver mais de um objetivo.
  - Formulação do problema: O agente deriva uma representação dos estados e das ações. Uma abstração é feita do problema a ser resolvido. Por exemplo: Considerar ações de viajar de uma cidade para cidades adjacentes. O mundo é representado por cidades, e a cidade atual muda conforme uma ação tomada.
  - Busca: Antes de tomar qualquer ação, o agente simula sequências de ações em seu modelo, até encontrar uma sequência que atinge o objetivo. Essa sequência é chamada de **solução**.
  - Execução: Depois que uma solução foi encontrada, o agente a executa no mundo, uma etapa por vez.

Looping aberto e fechado

- Se o ambiente é totalmente observável, determinístico, e conhecido, é seguro executar ignorar os sensores e executar uma ação por vez, pois sabemos que não haverá nenhuma interferência externa. Isso é conhecido como **looping aberto**.
- Se há alguma chance do modelo do mundo estar incorreto, ou o ambiente ser não determinístico, então deve-se monitorar os sensores a cada execução das ações da solução, pois algo pode interferir no resultado. Isso é conhecido como **looping fechado**.

A formulação dos problemas I

Um problema pode ser definido formalmente através de cinco propriedades:

1. Espaço de Estados: Um conjunto de possíveis estados que um ambiente pode estar.
2. Um estado inicial no qual um agente pode estar. Ex: Arad.
3. Um conjunto de um ou mais estados-objetivos (ex. Bucharest).
4. As ações disponíveis para o agente. Dado um estado s, $Acoes(s)$ retorna um conjunto de ações que podem ser aplicadas em s. Exemplo: $Acoes(Arad) = \{irSibiu, irTimisoara, irZerind\}$
5. Um modelo de transição: Define o que cada ação faz. $Resultado(s, a)$ retorna o estado resultante da ação a em s. Exemplo: $Resultado(Arad, irZerind) = Zerind$
6. Uma função de custo de ação. Denotado por $CustoAcao(s, a, s')$, que resulta em um custo numérico de aplicar a ação a, no estado s para atingir o estado s'. Essa função reflete a medida de performance.

A formulação dos problemas II

Uma sequência de ações é denominada de **caminho**. Uma **solução** de um problema é então um caminho que sai do estado inicial e atinge o estado-solução. A qualidade de uma solução é medida pelo custo de caminho, que consiste de uma métrica de custo interna do agente. Uma **solução ótima** tem o menor custo de caminho entre todas as soluções.

Modelos e abstrações

- No exemplo da aula passada, a sala foi descrita como um ambiente retangular e sem obstáculos. Isso não é nada parecido com cômodos de uma casa real, que podem ter formas geométricas variadas e objetos situados no percurso do agente.
- Para alguns ambientes, no entanto, essa formulação pode ser adequada (o dono do aspirador pode retirar os móveis de um cômodo para facilitar a operação do agente).
- A formulação do problema e o comportamento do agente são, portanto, modelos que podem ser úteis em determinadas situações, mas que **NUNCA** podem ser vistos como uma síntese da realidade.
- O processo de descartar detalhes de uma representação é chamado de **abstração**.

Uma frase para a vida

"Todos os modelos estão errados, mas alguns são úteis." -*George Box*

Um problema para praticar

- Problemas reais são aqueles de fato importantes para alguém. Geralmente, no entanto, podem ser muito difíceis e demandar esforços que não são cabíveis a uma disciplina de graduação.
- Um exemplo clássico de problema real e extremamente difícil é o do caixeiro viajante, no qual dada uma lista de cidades e distâncias entre estas deseja-se encontrar o menor caminho possível para visitar todas as cidades sem repetir nenhuma.
  - Encontrar um caminho capaz de realizar a tarefa pode ser até simples, mas a condição de optimalidade torna o problema computacionalmente intratável (dito *NP-Hard*).
- Nessa aula, vamos nos ater a um problema simples, mas que já é suficientemente complicado: *8-puzzle*.

Um problema para praticar

No jogo chamado *8-puzzle* número de 1 a 8 são distribuídos em um *grid* 3×3, restando um espaço vazio que permite o movimento das peças adjacentes. O objetivo é então ordenar os números de 1 a 8 conforme mostrado na figura.

3	1	2
4	7	5
	6	8

	1	2
3	4	5
6	7	8

Figura: Exemplo de uma configuração inicial (esq.) e da configuração meta para o *8-puzzle*.

Um problema para praticar

1. **Estados:** Um estado do jogo é descrito unicamente pela configuração das peças.
2. **Estado Inicial:** O estado inicial poderia, *a priori* ser qualquer um. No entanto, há questões combinatórias envolvidas: para um dado estado inicial, apenas metade das configurações possíveis é atingível a partir do mesmo. Logo, **existem configurações para as quais o jogo nunca pode ser vencido**.
3. **Ações:** As ações possíveis a um agente que deseja resolver o problema são os movimentos possíveis do espaço vazio.
4. **Modelo de Transição:** Para uma dada configuração do *grid*, o modelo de transição é o resultado de uma ação.
5. **Estado Objetivo:** A checagem do objetivo consiste em verificar se o *grid* está na configuração desejada.
6. **Custo da ação:** Cada movimento tem custo 1, então o custo de um caminho é igual ao número de passos no caminho.

Exercício sugerido

Faça a formulação do problema para o exemplo do aspirador bidimensional visto na última aula, descrevendo as 5 propriedades discutidas nesta aula.

O problema do aspirador

- Suponha o mundo do aspirador, no qual está em um mundo unidimensional com apenas 2 quartos. O problema pode ser descrito como:
- **Estados:** Onde o aspirador e as sujeiras estão no grid. O aspirador pode estar nas 2 posições, e há 4 configurações de sujeira, resultando em $2 * 2 * 2 = 8$ estados possíveis.
- **Estado inicial:** Qualquer estado pode ser o inicial.
- **Ações:** Esquerda, Direita e Limpar.
- **Modelo de transição:** Limpar remove a sujeira da posição atual. Esquerda e direita movem o agente nas respectivas posições.
- **Estado objetivo:** Estado no qual todas as células estão limpas.
- **Custo da ação:** Cada ação tem custo 1.

O problema do aspirador

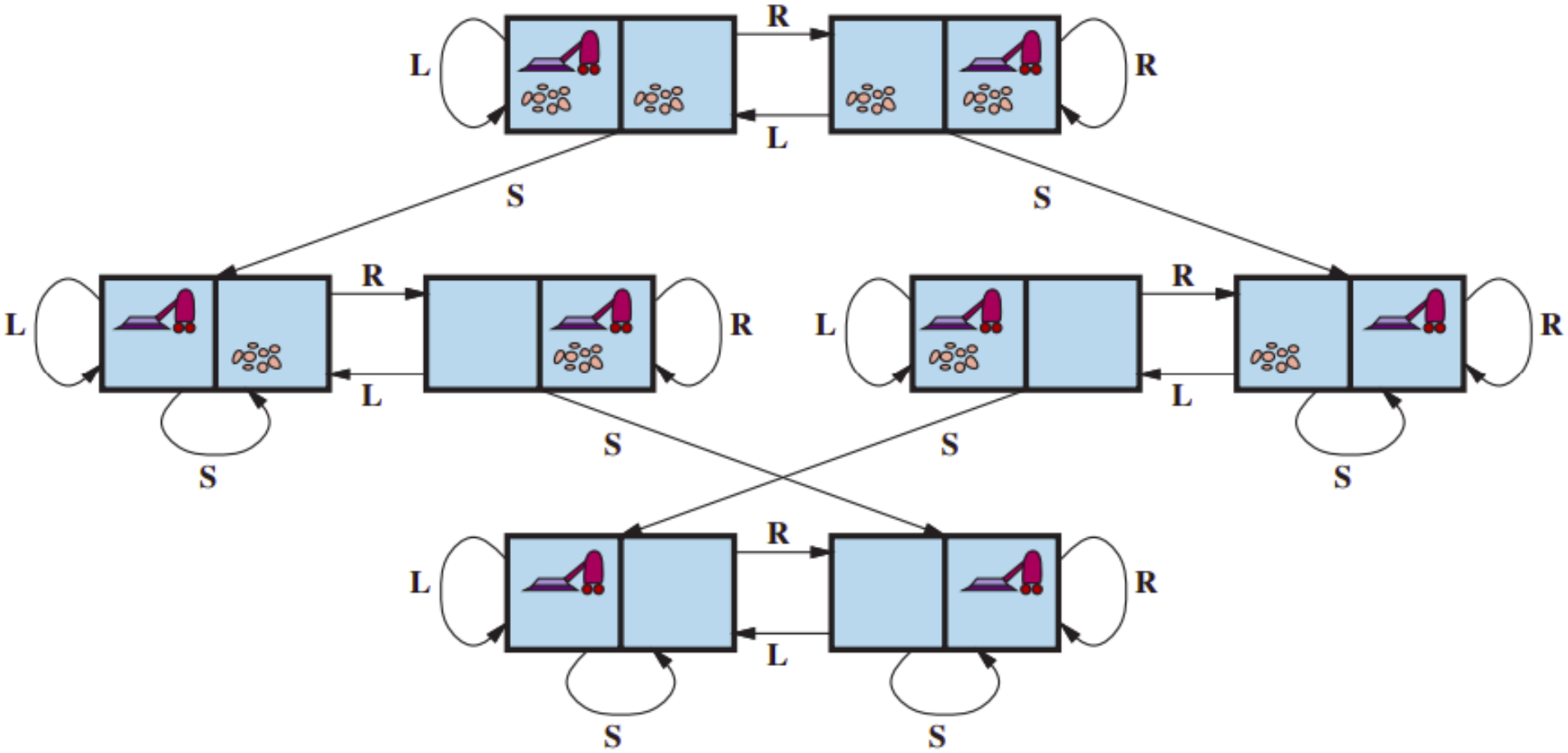

Figura: Modelo de transição do problema do aspirador.

Buscando por soluções

- Como dito anteriormente, o modelo de transição permite ver o espaço de estados como um grafo no qual as ações conectam os estados.
- Assim, a solução de um problema passa pela busca em um grafo: começando de um estado inicial, deseja-se encontrar um caminho até a solução.
  - No que encontrar a melhor solução pode tornar-se excepcionalmente difícil caso seja necessário checar todos os caminhos possíveis. Vamos nos ater a encontrar apenas uma solução do *8-puzzle*, sem demandar optimalidade desta.
- Vamos explorar primeiro métodos clássicos de **força-bruta** que, no pior dos casos, checam todos os caminhos possíveis.

Medidas de performance para algoritmos de busca

Para algoritmos de busca, existem quatro questões de interesse:

- **Completude:** o algoritmo consegue sempre encontrar uma solução quando esta existe?
- **Optimalidade:** o algoritmo consegue uma solução ótima?
- **Complexidade temporal:** quanto tempo leva para encontrar uma solução?
- **Complexidade de armazenamento:** quanta memória é necessária para encontrar uma solução?

Uma base para os algoritmos de busca

- O algoritmo de busca recebe um problema como entrada e retorna uma solução ou uma indicação de falha.
- Ele constrói uma **árvore de busca** sobre o espaço de estados. Não confundir um com o outro!
- Cada nó da árvore de busca representa um estado no espaço de estados.
- Cada vértice da árvore de busca representa uma ação.

Busca informada e não informada

- Se o agente não sabe o quão perto um estado está do objetivo, a busca é chamada de **não informada**.
- Caso contrário, a busca é chamada de **informada**.
- Exemplo: Se o agente sabe a distância em linha reta de cada cidade à Bucharest, então ele sabe que tomar o caminho das cidades mais próximas provavelmente levará ao objetivo mais rápido. Este caso seria uma busca informada.
- Se, a cada cidade, o agente não saber a distância do alvo, se não houver informações sobre coordenadas, distância em linha reta, nenhuma informação que informe o quão longe o objetivo está, então a busca é não informada.

Árvore de busca para o problema da Romênia

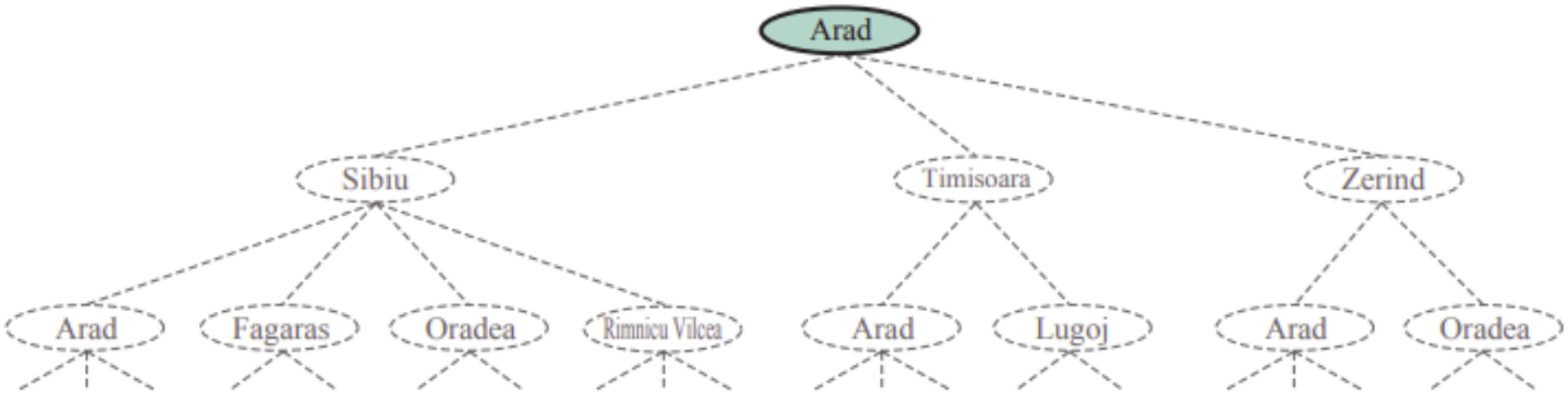

Figura: Árvore de busca. A raiz é o estado inicial. Os nós roxos são os estados já visitados. Os nós azuis são a fronteira. Nós brancos são estados ainda não visitados.

Árvore de busca para o problema da Romênia

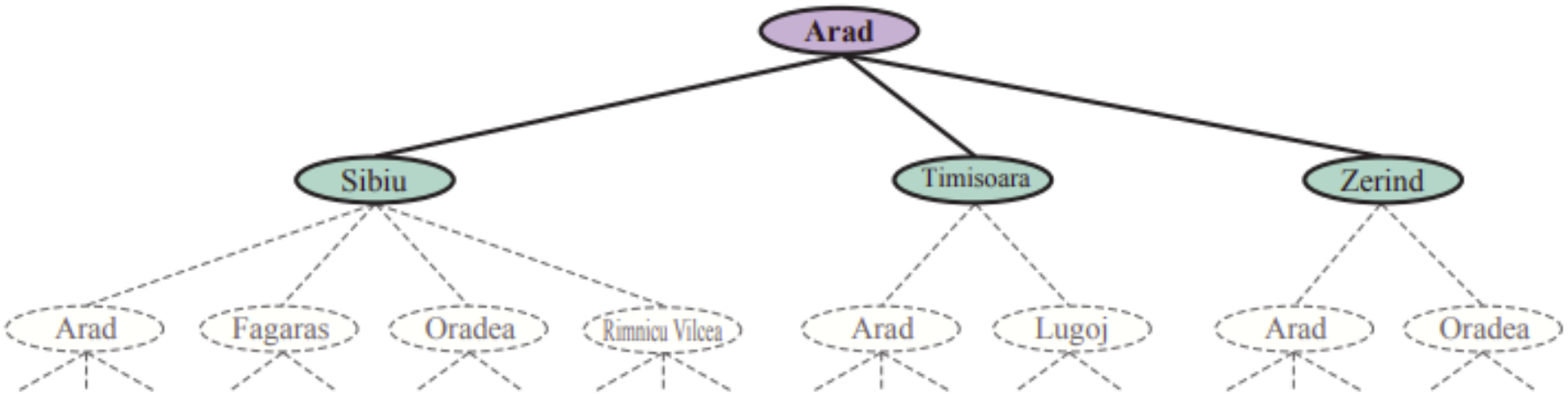

Figura: Árvore de busca. A raiz é o estado inicial. Os nós roxos são os estados já visitados. Os nós azuis são a fronteira. Nós brancos são estados ainda não visitados.

Árvore de busca para o problema da Romênia

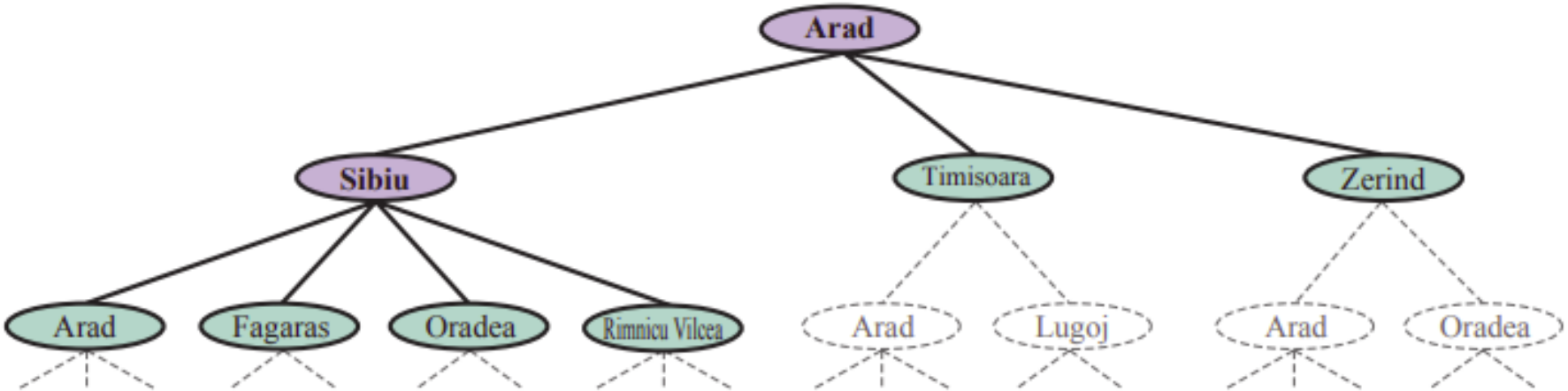

Figura: Árvore de busca. A raiz é o estado inicial. Os nós roxos são os estados já visitados. Os nós azuis são a fronteira. Nós brancos são estados ainda não visitados.

Busca em largura

- A ideia da busca em largura consiste de expandir (isto é, gerar filhos) dos nós em uma mesma altura da árvore de busca.
- Há dois critérios de parada para o algoritmo:
  - O estado buscado é encontrado e então retorna-se o nó que armazena o mesmo.
  - Todos os estados são explorados e não encontra-se o estado buscado.

Busca em largura - a implementação

- Ao fazer a expansão de um nó, seus filhos são inseridos em uma estrutura de **fila**
- Uma fila é uma lista com uma dinâmica do tipo **FIFO** (*First In First Out*), na qual os nós que foram inseridos primeiro, são retirados primeiro. Na implementação discutida, são necessárias apenas algumas das funções desta estrutura de dado:
  - *insert*, que insere um nó na última posição da fila.
  - *pop*, que retorna o nó que foi inserido primeiro, removendo-o da fila.
  - *isIn*, que checa se um nó já foi inserido na fila, de forma a evitar a expansão de um estado já visitado.

Busca em largura - o algoritmo

Busca em largura

Entrada: estado inicial S_0. **Saída:** nó que resolve o problema ou erro.

- nd $\leftarrow$ nóFilho(NULL,S_0,NULL)
- **Se** S_0 é a meta **então** retorna nd
- Inicializa duas filas: F (fronteira) e E (explorados) e adiciona nd a ambas
- **Enquanto** F não estiver vazia, faça
  - p = $pop(F)$
  - **Para cada** ação A possível no estado p.S
    - filho = nóFilho(p,p.S,A)
    - **Se não** $isIn(E, filho)$:
      $\rightarrow$ **Se** filho.S é a meta **então** retorna filho
      $\rightarrow$ $insert(F$,filho), $insert(E$,filho)
- $Print$("Estado não encontrado")

Busca em largura - o algoritmo I

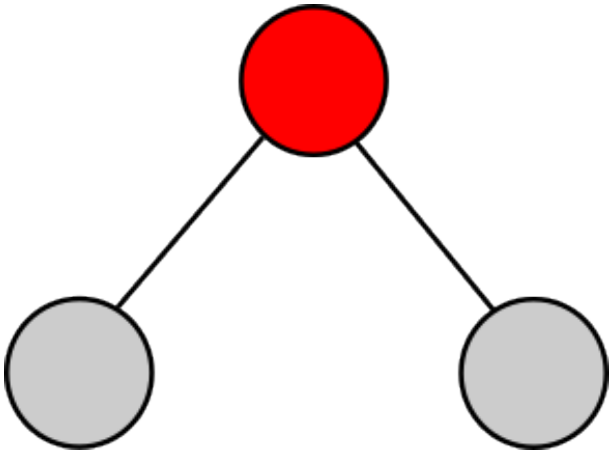

Figura: Primeira expansão de uma busca em largura.

Busca em largura - o algoritmo II

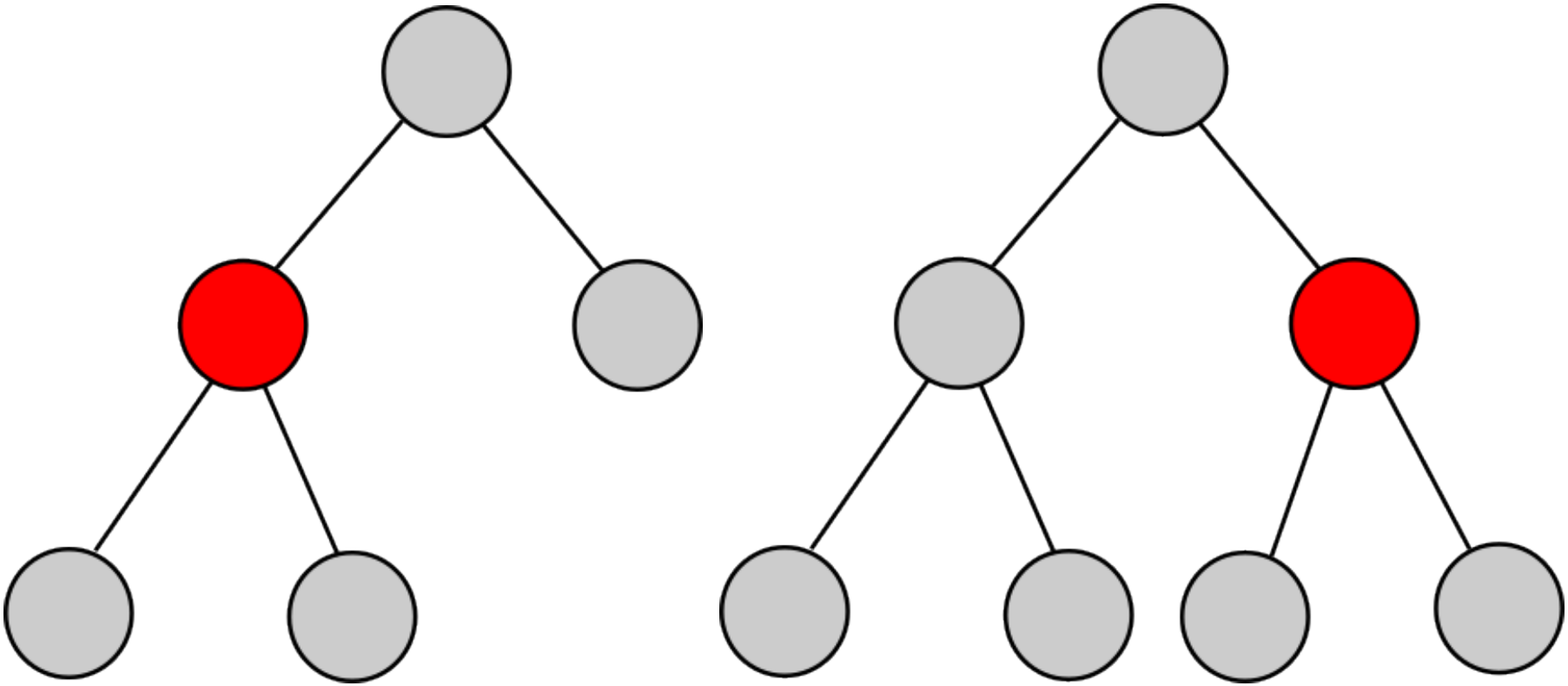

Figura: Expansão dos dois primeiros filhos.

Busca em largura - o algoritmo III

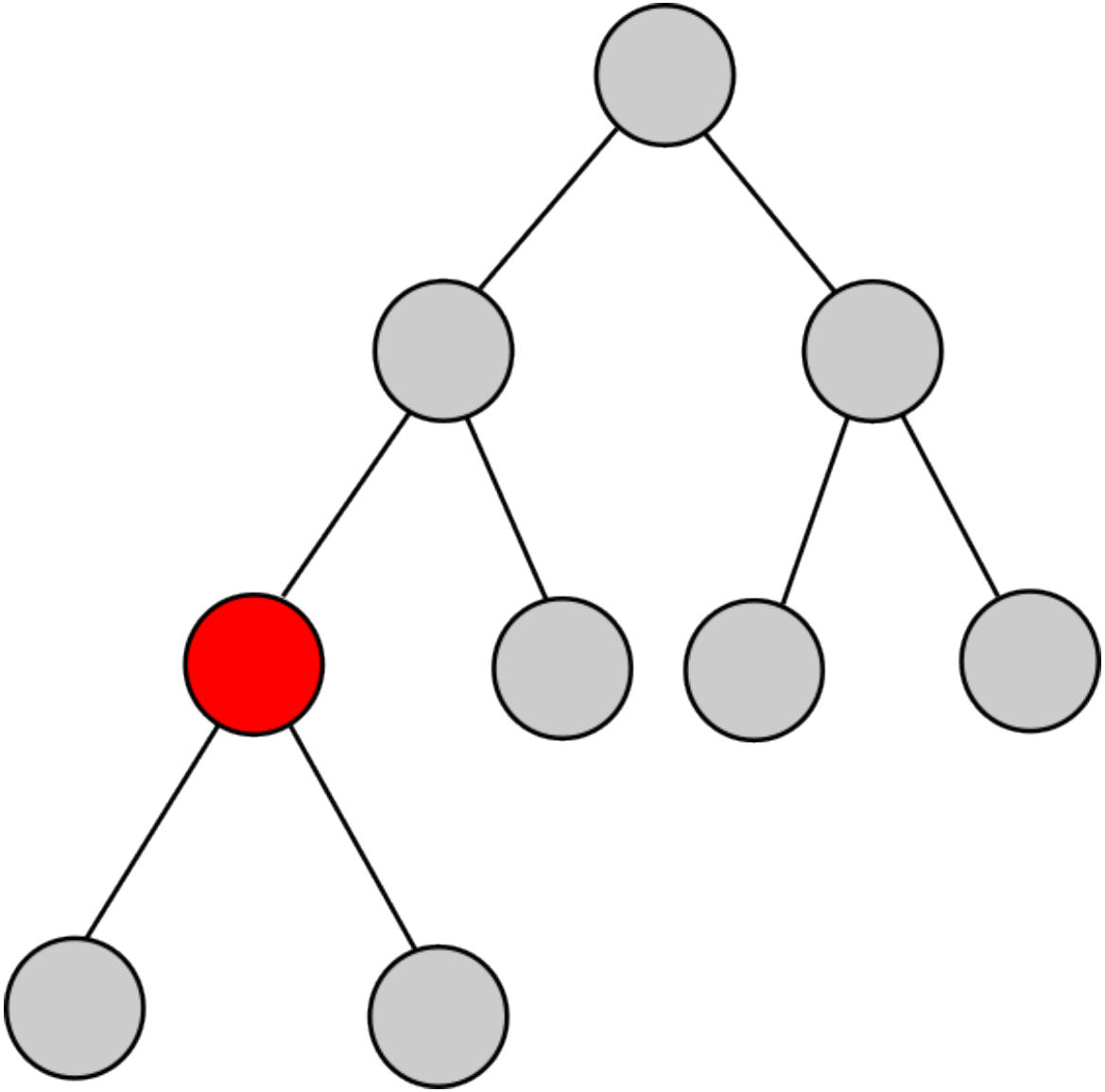

Figura: E assim sucessivamente.

Busca em largura - caminhos redundantes

- A busca pelo mapa da Romênia pode entrar em looping.
- Por exemplo, existe um caminho de Arad para Sibiu, e um outro voltando, de Sibiu para Arad.
- Chamamos esse caso de **ciclo**, ou **caminho tortuoso**.
- **Solução 1:** No algoritmo anterior, resolvemos isso inserindo uma lista de explorados, que lembra todas as cidades já visitadas. Essa solução só é viável quando a lista de explorados cabe na memória. Nesses caso, chamamos o algoritmo de **busca em grafo**.
- **Solução 2:** Há algumas soluções que ciclos são impossíveis, e então não precisamos detectá-los. Nesses caso, chamamos o algoritmo de **busca em árvore**.
- **Solução 3:** É possível checar a existência de ciclos por meio de um ponteiro para o nó pai.

Busca em largura - a análise I

- O número de filhos gerados por cada nó é dito o fator de ramificação do problema e será denotado por b.
- Assim, a primeira expansão gera b filhos, enquanto a segunda gerará b^2, a terceira b^3 e, para um nível d, b^d filhos serão gerados.
- Assim, o custo computacional para chegar ao nível d da árvore de busca é

$$K = \sum_{i=1}^{d} b^i \quad \rightarrow \quad K \in O(b^d) \tag{1}$$

- Aqui, d representa a quantidade passos necessários para atingir a meta de busca e, caso seja suficientemente grande, pode demandar uma quantidade de tempo e memória exorbitante para resolução do problema.

Busca em largura - a análise

- O algoritmo é completo? Sempre encontra uma solução quando existe ou reporta falha se não existir?
- **SIM**, pois ele avalia todos os estados
- **E em estados infinitos?**
- **SIM,** pois ele é **sistemático**. Todos os nós de distância b da origem são avaliados, depois de distância $b + 1$, e assim por diante.
- Mas ele reporta falha se não existir? **NÃO**. Mas nenhum método iria reportar. Mesmo assim consideramos a busca por largura como **COMPLETO**.

Busca em largura - a análise

- Busca em largura pode demandar uma quantidade exorbitante de memória em função do tamanho do espaço de estados. Isso acaba sendo um impeditivo maior que o tempo de execução, pois é mais viável deixar um computador ligado por um longo período de tempo, mas raramente será possível conseguir mais memória para uma máquina, especialmente se a demanda por memória cresce exponencialmente a cada nível da árvore.

Algoritmo Dijkstra (Busca de Custo Uniforme)

- A busca por largura não considera os custos dos caminhos (ou considera todos com custo 1).
- Uma melhoria seria priorizar os caminhos mais curtos (distância da origem até o nó atual).
-

Busca de custo uniforme (Dijkstra) - o algoritmo

Dijkstra

Entrada: estado inicial S_0. **Saída:** nó que resolve o problema ou erro.

- nd $\leftarrow$ nóFilho(NULL,S_0,NULL)
- nd.custo $\leftarrow$ 0
- **Se** S_0 é a meta **então** retorna nd
- Inicializa:
  - F como fila de prioridade (ordenada por custo)
  - E como conjunto de explorados
- *insert*(F,nd)
- **Enquanto** F não estiver vazia, faça
  - p = *pop*(F) // menor custo
  - **Se** *isIn*(E, p) **continue**
  - *insert*(E, p)
  - **Se** p.S é a meta **então** retorna p
  - **Para cada** ação A possível no estado p.S
    - filho = nóFilho(p,p.S,A)
    - filho.custo $\leftarrow$ p.custo + custo(A)
    - *insert*(F,filho)
- *Print*("Estado não encontrado")

Diferenças: BFS vs Dijkstra

BFS	**Dijkstra**
Fila FIFO	Fila de prioridade
Não usa custo	Usa custo acumulado
Ótima só com custos iguais	Sempre ótima (custos positivos)
Marca explorado na inserção	Marca ao expandir
Testa meta na geração	Testa meta na expansão

Exemplo

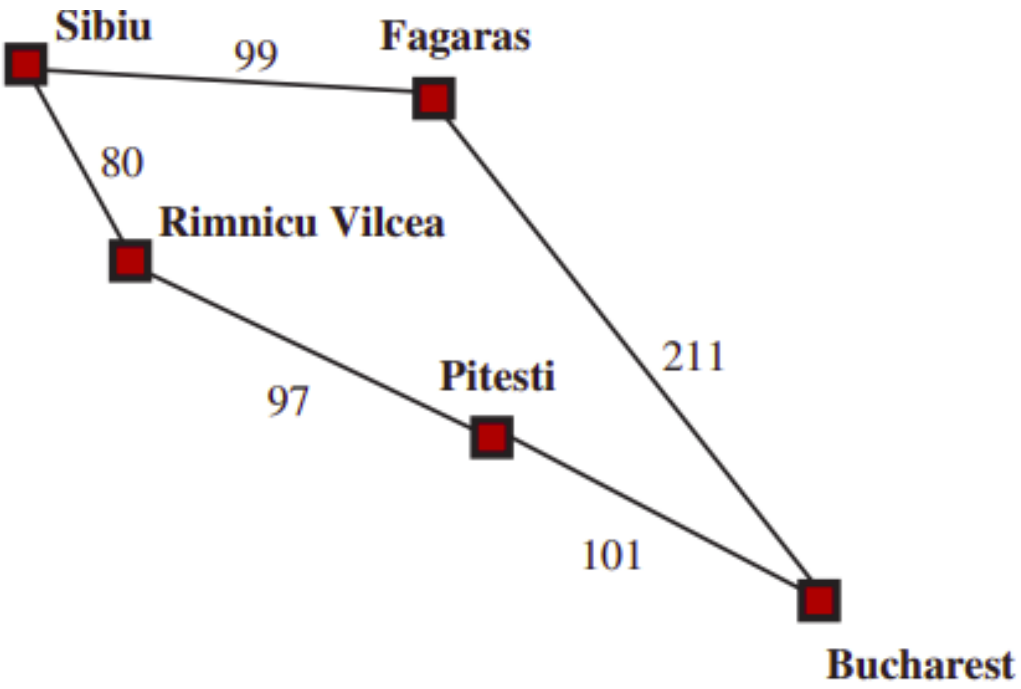

Figura: Parte do mapa da Romênia

Dijkstra - Propriedades

- Complexidade do pior caso (tempo e espaço): $O\left(b^{1+\lfloor C^*/\epsilon \rfloor}\right)$.
- C^* é o custo da solução ótima. ϵ é o menor custo de ação do problema.
- É completo e ótimo

Busca em Profundidade

- A Busca em Profundidade expande o nó **mais profundo primeiro**.
- É implementado usando uma **pilha**.

Busca em profundidade (DFS) - o algoritmo

Busca em profundidade

Entrada: estado inicial S_0. **Saída:** nó que resolve o problema ou erro.

- nd $\leftarrow$ nóFilho(NULL,S_0,NULL)
- **Se** S_0 é a meta **então** retorna nd
- Inicializa:
  - F como pilha (LIFO)
  - E como conjunto de explorados
- *insert*(F,nd)
- **Enquanto** F não estiver vazia, faça
  - p = *pop*(F)
  - **Se** *isIn*(E, p) **continue**
  - *insert*(E, p)
  - **Se** p.S é a meta **então** retorna p
  - **Para cada** ação A possível no estado p.S
    - filho = nóFilho(p,p.S,A)
    - *insert*(F,filho)
- *Print*("Estado não encontrado")

Um exemplo da busca em profundidade

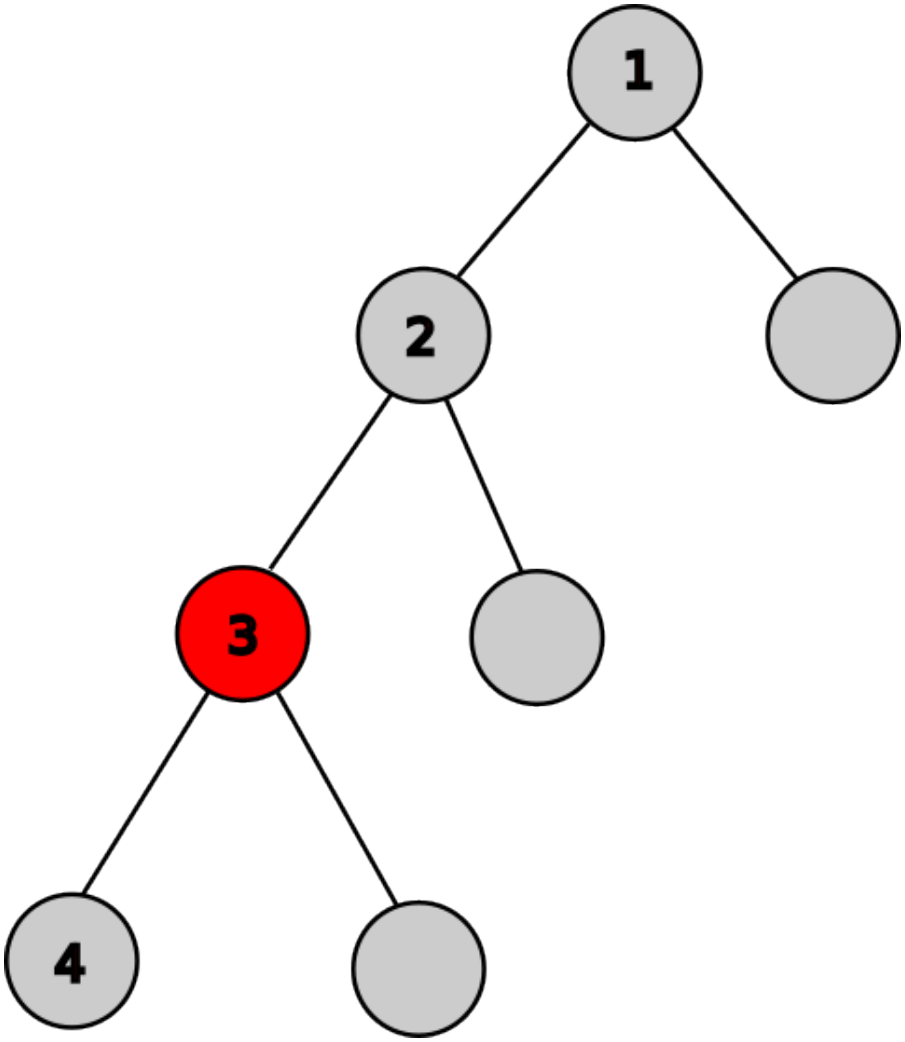

Figura: Exemplo visual do algoritmo de busca em profundidade.

Características

- Usa pilha (LIFO): explora um caminho até o fundo
- Teste de objetivo ocorre na expansão
- Pode entrar em loop infinito (se não controlar ciclos). Não é o caso do algoritmo mostrado, pois estamos evitando os ciclos usando a lista E.
- Não é ótima em termos do custo, pois retorna a primeira solução encontrada, mesmo não sendo a melhor delas.
- Para espaço de estados finitos, é completo.
- Para espaço de estados infinitos, a busca não é sistemática, portanto, o algoritmo se torna incompleto.

Características

- Usa menos memória que BFS (grande vantagem). DFS só guarda (i) o caminho atual e (ii) alguns irmãos pendentes.
- Por isso a memória cresce linearmente com a profundidade $O(b * m)$, sendo $b =$ fator de ramificação e $m =$ profundidade máxima da árvore.
- Em relação a complexidade tempora, no pior dos casos, toda a árvore de busca deve ser explorada, chegando a $O(b^m)$

Busca Heurística (Informada)

- Nos algoritmos de busca clássica a questão essencial é achar uma forma de explorar o espaço de busca de forma eficiente, independentemente das especificidades do problema.
- A busca heurística ou busca informada utiliza-se de propriedades do problema de forma a guiar o algoritmo pelo espaço de busca.
- Em espaços de estados grandes demais, esta pode ser a única forma possível de realizar uma busca que retornará algum resultado em um tempo hábil.
- De forma geral, nossa abordagem será a de procurar pelas "melhores soluções"de acordo com alguma métrica e usá-las para orientar o caminho de busca.

Busca heurística

Função heurística

Uma função heurística é uma estimativa do menor custo possível para ir do estado atual para o estado meta do problema.
Será aqui considerada uma estratégia chamada de **melhor-primeiro**, na qual a cada nó expandido escolhe-se o filho mais próximo do objetivo de acordo com uma dada heurística $h(n)$, sendo n um nó filho.

Busca A*

- Nesta estratégia, o custo estimado corresponde ao custo de caminho adicionado da heurística $h(n)$:

$$f(n) = g(n) + h(n). \tag{2}$$

- Esta abordagem é optimal (em árvores) se a heurística $h(n)$ for **admissível**, que significa que esta **nunca superestima o custo de alcançar a meta**.
- Para optimalidade em grafos, uma condição suficiente é a de **consistência**, na qual, para todo nó n e todo filho n' gerado por uma ação A, a desigualdade triangular abaixo é satisfeita:

$$h(n) \leq c(n, A, n') + h(n'). \tag{3}$$